

PHASE 13 — PQC RECOMMENDATION ENGINE & CRYPTO-AGILITY

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

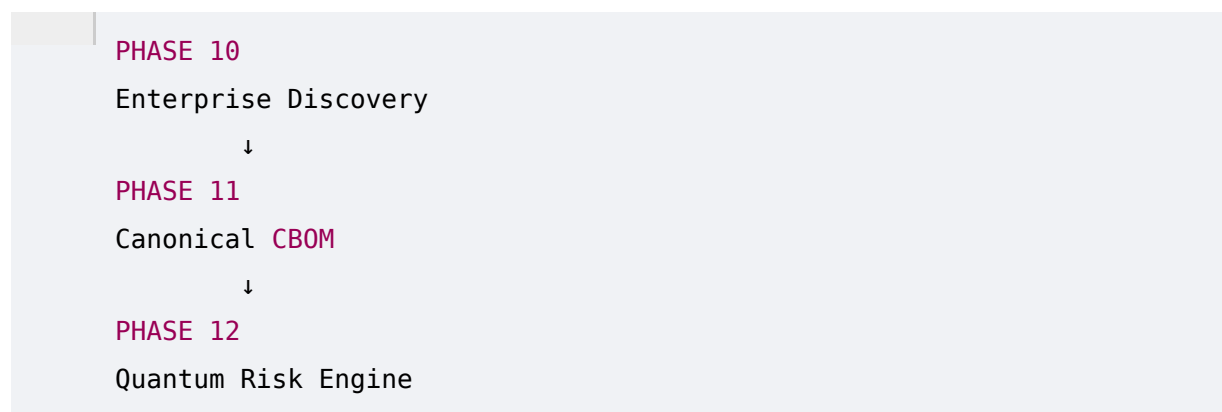
Theme: Blockchain & Cybersecurity

Phase: 13 of 22

Status: Post-Quantum Recommendation & Migration Decision Architecture

1. Purpose of This Phase

The previous phases established:



ECDAT can now answer:

| What cryptography exists?

and:

| What is at risk?

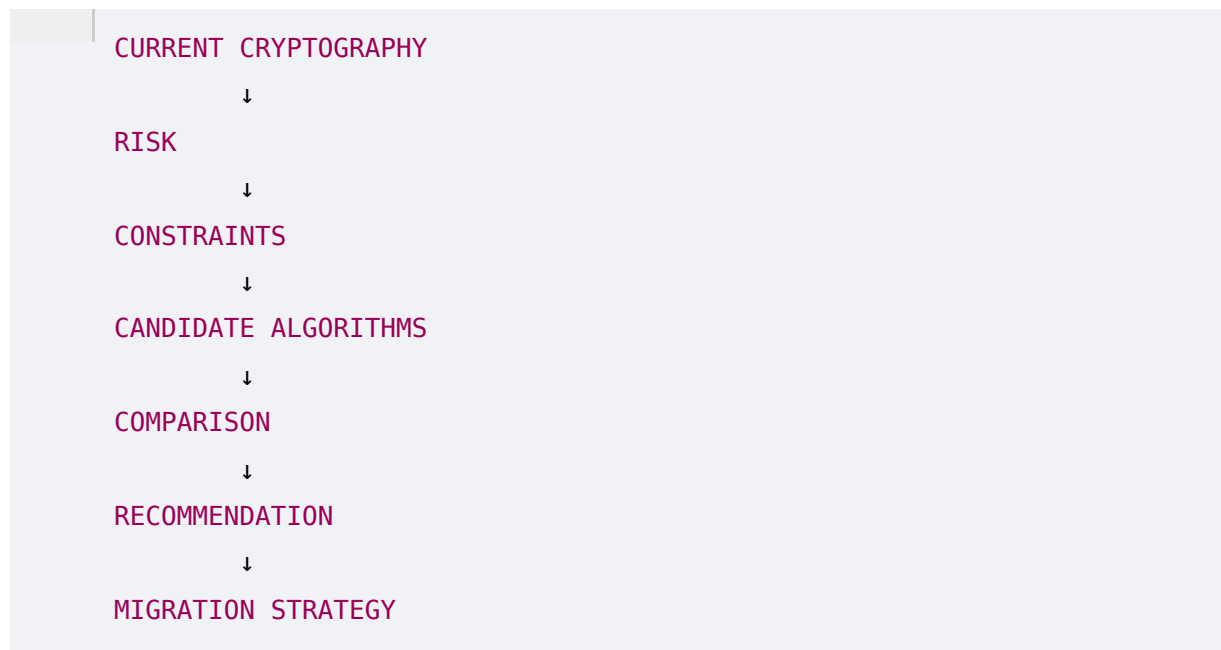
But the SIH problem statement requires something much more valuable:

| **Recommend suitable alternatives (PQC/Hybrid algorithms) for applications based on risk profile, latency, cost, etc.**

Therefore Phase 13 builds the:

PQC Recommendation Engine

Its responsibility is to transform:



2. The Core Problem

It is tempting to build a simple mapping:

```
RSA → ML-DSA
ECDSA → ML-DSA
ECDH → ML-KEM
```

That is **not enough**.

Real enterprise systems have constraints.

For example:

```
Application A
├─ RSA
├─ Legacy TLS
├─ HSM dependency
├─ 10 ms latency requirement
├─ 100,000 requests/sec
└─ Legacy clients
```

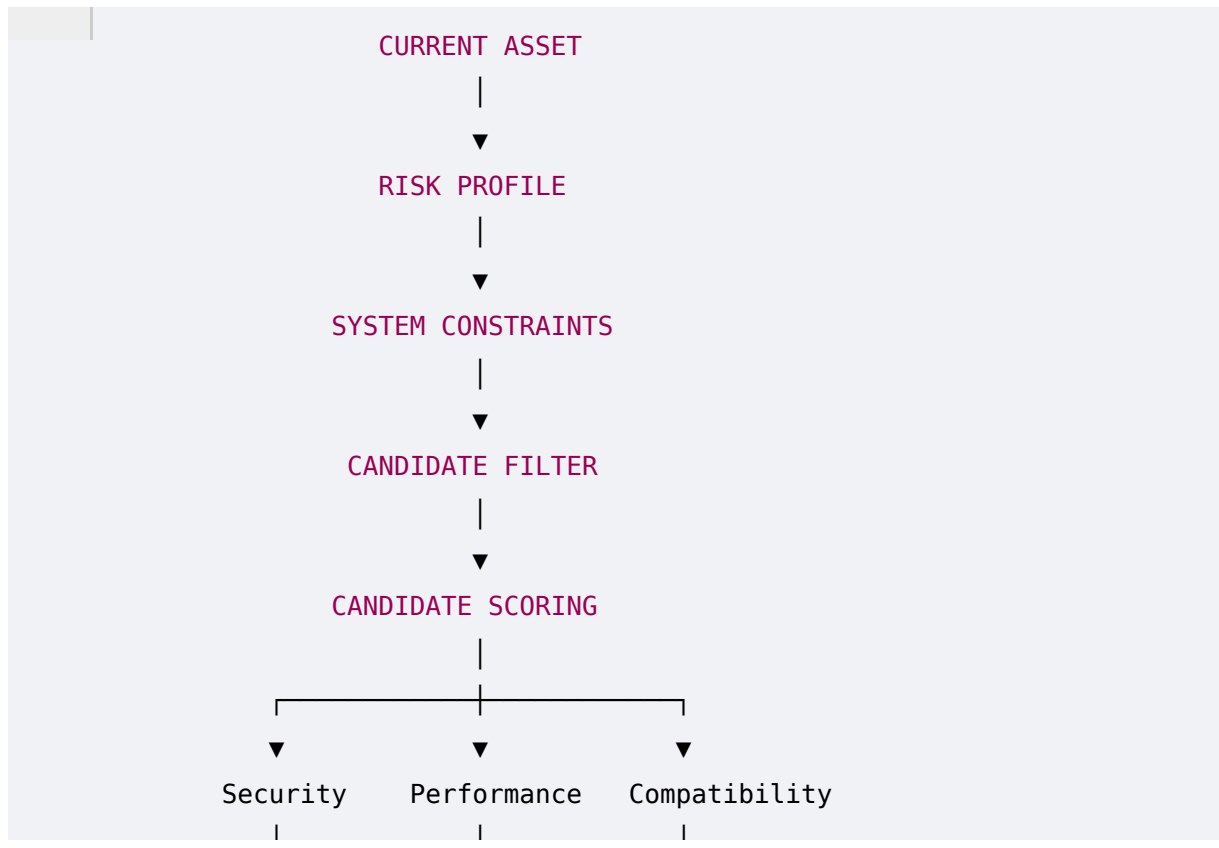
A recommendation must consider all of these.

Therefore:

PQC migration is an optimization problem, not a lookup table.

3. Recommendation Architecture

The recommendation engine should operate as:



4. What Does "Suitable" Mean?

A PQC algorithm is suitable only if it satisfies the application's requirements.

ECDAT should consider:

- Security
- Performance
- Latency
- Throughput
- Memory
- Bandwidth
- Key Size
- Signature Size
- Compatibility
- Library Support
- Hardware Support
- Protocol Support
- Compliance
- Cost
- Migration Complexity
- Interoperability

5. PQC Algorithm Categories

ECDAT should distinguish between two major families.

Key Establishment

Used to establish shared secrets.

Primary category:

KEM

Digital Signatures

Used for:

Authentication
Integrity
Non-repudiation
Certificates
Software Signing

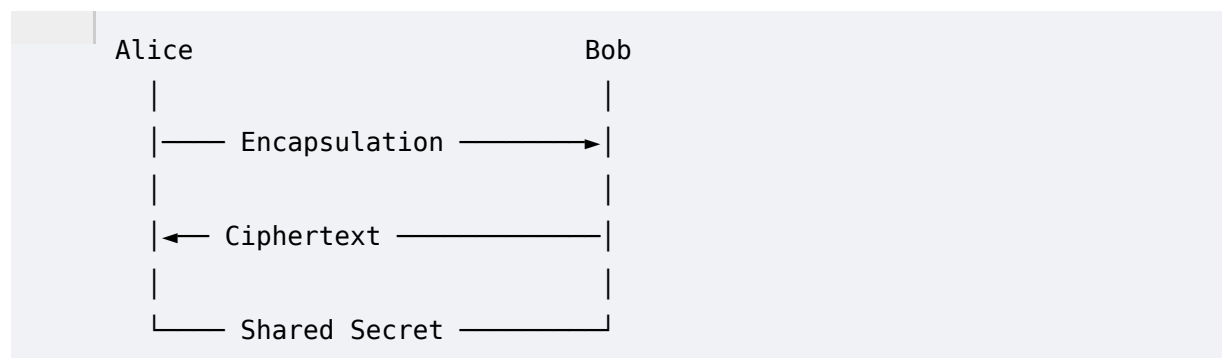
These require a different algorithm family.

6. KEM Architecture

KEM means:

Key Encapsulation Mechanism

Conceptually:



The shared secret can then be used with symmetric cryptography.

7. ML-KEM

A major PQC KEM family is:

ML-KEM

ECDAT should represent:

```
Algorithm:
ML-KEM

Category:
KEM

Purpose:
Key Establishment

Quantum Posture:
PQC
```

It should not simply store:

```
ML-KEM = encryption
```

because technically it is a key encapsulation mechanism rather than a general-purpose bulk encryption algorithm.

8. Digital Signatures

Digital signatures are used for:

```
Authentication
Software Signing
Certificates
Document Signing
Code Signing
Identity
```

ECDAT should maintain a separate recommendation pipeline.

9. ML-DSA

ML-DSA is a major standardized PQC digital-signature family.

ECDAT can consider it for:

```
ECDSA
RSA signatures
DSA
```

depending on the exact application and protocol.

10. SLH-DSA

SLH-DSA is another PQC signature family.

It provides a different design tradeoff from ML-DSA.

ECDAT should therefore treat:

```
ML-DSA
SLH-DSA
```

as candidate algorithms rather than assuming one is universally superior.

11. Algorithm Candidate Registry

ECDAT should maintain a registry:

```
PQC Algorithm Registry
|
├─ ML-KEM
├─ ML-DSA
├─ SLH-DSA
└─ Future Algorithms
```

This registry should be versioned.

12. Why Versioning Matters

Cryptographic standards evolve.

Therefore ECDAT should track:

```
Algorithm
Standard
Version
Parameter Set
Status
```

Approval Date
Deprecation Status

This prevents recommendations from becoming stale.

13. Candidate Algorithm Object

Conceptually:

```
{  
  "algorithm": "ML-DSA",  
  "category": "signature",  
  "status": "approved",  
  "security_level": 3,  
  "key_size": "...",  
  "signature_size": "...",  
  "performance": "...",  
  "hardware_support": "...",  
  "protocol_support": "..."  
}
```

The exact values should come from maintained authoritative technical data rather than hardcoded assumptions.

14. Recommendation Is Contextual

Consider:

ECDSA P-256

Candidate:

ML-DSA

That does not automatically mean:

REPLACE IMMEDIATELY

The engine must ask:

Is the protocol compatible?
Does the library support it?
Does the HSM support it?

Can clients verify it?
Can certificates use it?
What happens to signature size?

15. Hybrid Cryptography

One of the most important transition concepts is:

Hybrid Cryptography.

Instead of immediately replacing classical cryptography:

Classical

the system can use:

Classical + PQC

during migration.

16. Why Hybrid Approaches Matter

Enterprises often have legacy systems.

For example:

Client A:
Classical only

Client B:
PQC capable

Server:
Needs to support both

A hybrid architecture can provide transition flexibility.

17. Hybrid KEM

Conceptually:

Classical KEM
+
ML-KEM

↓
Combined Key Material
↓
Symmetric Key

The precise construction depends on the protocol and standards being used.

ECDAT should not invent its own cryptographic composition.

18. Hybrid Signatures

Similarly:

Classical Signature
+
PQC Signature
↓
Hybrid Authentication

Again, the exact construction must follow approved standards or protocol specifications.

19. Migration Modes

ECDAT can recommend:

DIRECT_REPLACEMENT
HYBRID
PARALLEL
UPGRADE
RECONFIGURE
WRAP
DECOMMISSION
MONITOR

20. Direct Replacement

Example:

Old:
ECDSA

```
New:  
ML-DSA
```

Appropriate when:

- Compatibility is available
 - Legacy clients are not a constraint
 - PQC support is mature
-

21. Hybrid Migration

Example:

```
Old:  
ECDSA  
  
Transition:  
ECDSA + ML-DSA  
  
Future:  
ML-DSA
```

Useful when interoperability requires gradual transition.

22. Parallel Migration

Two systems may temporarily coexist:

```
System A:  
Classical  
  
System B:  
PQC  
  
Migration:  
Traffic gradually moved
```

Useful for large infrastructure.

23. Upgrade

Sometimes the correct recommendation is not to change the application directly.

Instead:

```
Application
  ↓
Crypto Library
  ↓
Upgrade Library
  ↓
Enable PQC Capability
```

This may be significantly safer.

24. Reconfigure

Some migrations require only configuration changes.

Example:

```
Current:
Weak / Legacy Cipher

Target:
Approved Cipher
```

No major code rewrite may be necessary.

25. Decommission

An important insight:

The best migration is sometimes removing the cryptography entirely.

Example:

```
Legacy Application
  ↓
No longer used
  ↓
Decommission
```

ECDAT should be able to recommend this where evidence supports it.

26. Recommendation Inputs

The engine should receive:

```
Current Algorithm
Algorithm Parameters
Application
Data
Business Criticality
Risk
Protocol
Library
Hardware
Clients
Performance Requirements
Latency Requirements
Compliance
Migration Complexity
```

27. Security Constraint

Security should be a hard constraint.

For example:

```
Candidate:
Deprecated Algorithm
```

should not be recommended merely because:

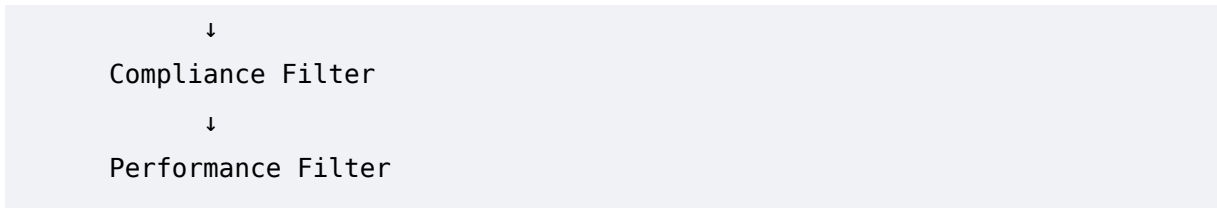
```
Latency = excellent
```

Security policy must eliminate unsuitable candidates first.

28. Candidate Filtering

First:

```
All Algorithms
  ↓
Security Filter
  ↓
Protocol Filter
  ↓
Hardware Filter
```



Only viable candidates reach scoring.

29. Candidate Scoring

Suppose:

ML-DSA
SLH-DSA
Hybrid

The engine can score them across:

Security
Performance
Compatibility
Cost
Migration Difficulty

Example:

Candidate	Security	Performance	Compatibility	Migration
ML-DSA	9	8	8	8
SLH-DSA	9	6	6	7
Hybrid	10	7	10	9

These numbers should be generated from documented evidence and configurable scoring policies.

30. Weighted Recommendation

A conceptual score:

$$[S = w_s S_s + w_p S_p + w_c S_c + w_m S_m + w_k S_k]$$

where:

- (S_s) = security
- (S_p) = performance
- (S_c) = compatibility
- (S_m) = migration feasibility
- (S_k) = cost

Weights are configurable.

31. Why Weights Matter

A:

High-frequency trading system

may prioritize:

Latency

while:

Long-term archival system

may prioritize:

Long-term security

and:

Government **PKI**

may prioritize:

Interoperability

+

Compliance

+

Long-term assurance

The engine must support policy profiles.

32. Policy Profiles

ECDAT could provide:

```
DEFAULT
HIGH_SECURITY
LOW_LATENCY
GOVERNMENT
FINANCIAL
CLOUD
LEGACY
ARCHIVAL
```

Each profile adjusts weights.

33. Performance Considerations

PQC algorithms can have different:

```
Key Sizes
Ciphertext Sizes
Signature Sizes
CPU Requirements
Memory Requirements
```

ECDAT should include these in recommendations.

34. Latency

For latency-sensitive systems:

```
Request
↓
Crypto Operation
↓
Response
```

the recommendation engine should evaluate the performance impact.

Example:

```
Current:
2 ms
```

Candidate:

5 ms

If the SLA is:

< 3 ms

the candidate may be unsuitable without architectural changes.

35. Throughput

For high-volume systems:

100,000 operations/sec

ECDAT should evaluate:

Operations/sec

CPU utilization

Concurrency

Scaling requirements

36. Memory

PQC algorithms can have different memory characteristics.

The recommendation engine should account for:

RAM

Stack

Heap

HSM memory

Embedded constraints

This is particularly important for:

IoT

Embedded

Edge

Firmware

37. Bandwidth

Signature and ciphertext sizes matter.

For example:

```
TLS handshake
Certificate exchange
API authentication
```

larger cryptographic objects can affect:

```
Network latency
Bandwidth
Packet fragmentation
Storage
```

ECDAT should therefore consider bandwidth constraints.

38. Key Size

ECDAT should compare:

```
Current Key
Candidate Key
```

where applicable.

But it must distinguish:

```
Public Key
Private Key
Ciphertext
Signature
Shared Secret
```

because these have different operational implications.

39. Signature Size

Signature size can influence:

```
Network
Storage
Certificate Size
Blockchain Transactions
```

This is particularly relevant to the SIH theme's blockchain/cybersecurity context.

40. Blockchain Example

Suppose a blockchain system currently uses:

ECDSA

Every transaction contains a signature.

A PQC signature may be substantially larger.

Therefore migration must consider:

- Block Size
- Transaction Size
- Bandwidth
- Storage Growth
- Validation Cost

ECDAT could flag:

- Migration Constraint:
- Large Signature Overhead

rather than blindly recommending replacement.

41. HSM Compatibility

A major enterprise constraint is hardware.

ECDAT should know:

- HSM Vendor
- Model
- Firmware
- Supported Algorithms
- PQC Support
- Hybrid Support
- Upgrade Path

42. Library Compatibility

Example:

```
Application
↓
OpenSSL Version
↓
PQC capability
```

If the deployed version cannot support the required algorithm:

```
Recommendation:
Upgrade crypto library first.
```

43. Protocol Compatibility

The recommendation engine must understand whether the target algorithm can actually be used by the relevant protocol.

Examples:

```
TLS
SSH
IPsec
S/MIME
PKI
Application Protocol
```

A theoretically suitable algorithm is useless if the protocol stack cannot carry it.

44. Client Compatibility

Suppose:

```
Server:
PQC ready

Clients:
Legacy only
```

Immediate migration may break interoperability.

ECDAT should therefore identify:

```
Compatibility Blocker:
Legacy Clients
```

45. Vendor Dependency

Some enterprise systems depend on:

```
Vendor appliance
```

If the vendor does not support PQC:

```
Migration blocker:  
Vendor support
```

The recommendation could become:

```
Hybrid where supported  
+  
Vendor upgrade tracking
```

46. Compliance

Organizations may have requirements involving:

```
Approved Algorithms  
Cryptographic Modules  
Certification  
Industry Standards  
Internal Security Policy
```

ECDAT should allow policy constraints.

Example:

```
Candidate:  
Algorithm X  
  
Policy:  
Not Approved  
  
Result:  
Rejected
```

47. Cost

Migration cost can include:

```
Software
Hardware
Licensing
Engineering
Testing
Certification
Downtime
Operations
Training
```

The recommendation engine can use:

```
Low
Medium
High
Very High
```

or an organization's actual cost model.

48. Migration Complexity

A simple model:

```
Complexity =
Code
+
Dependencies
+
Hardware
+
Protocol
+
Testing
+
Operations
```

The engine can estimate:

```
Easy
Moderate
Complex
Very Complex
```

49. Recommendation Confidence

A recommendation should have confidence.

Example:

Recommendation:

ML-DSA

Confidence:

0.94

Why?

Algorithm mapping:

Strong

Library support:

Confirmed

Protocol:

Confirmed

Hardware:

Confirmed

Client compatibility:

Partial

50. Recommendation Explanation

ECDAT should explain:

Why this algorithm?

Example:

ML-DSA is recommended because the current ECDSA deployment is quantum-vulnerable, the application supports the required cryptographic library version, latency constraints are acceptable, and the organization has identified a long-term confidentiality requirement.

51. Alternative Recommendations

ECDAT should not return only one option.

Example:

PRIMARY
ML-DSA
ALTERNATIVE
SLH-DSA
TRANSITION
ECDSA + ML-DSA

This allows human decision-making.

52. Recommendation Comparison

Example:

Criterion	ML-DSA	SLH-DSA	Hybrid
PQC posture	Strong	Strong	Strong
Performance	High	Lower	Medium
Signature size	Moderate	Larger	Larger
Legacy compatibility	Medium	Medium	High
Migration complexity	Medium	Medium	Lower

Recommended	★	Alternative	Transition
-------------	---	-------------	------------

The actual values should be based on measured/maintained benchmark data.

53. Recommendation Rules

ECDAT should support deterministic rules.

Example:

```

IF
  algorithm IN {RSA, ECDSA, ECDH}
AND
  data_lifetime > threat_horizon
AND
  migration_time > 0

THEN
  PQC_MIGRATION_REQUIRED

```

Rules provide predictable baseline behavior.

54. AI-Assisted Recommendations

AI can enrich the rule engine.

Example:

```

Rule Engine
  ↓
Candidate Options
  ↓
AI
  ↓
Contextual Explanation

```

AI should not override hard security policies.

55. AI's Role

Good AI tasks:

- Explain migration impact
- Identify hidden dependencies
- Interpret source code
- Suggest migration sequence
- Summarize constraints
- Generate developer guidance

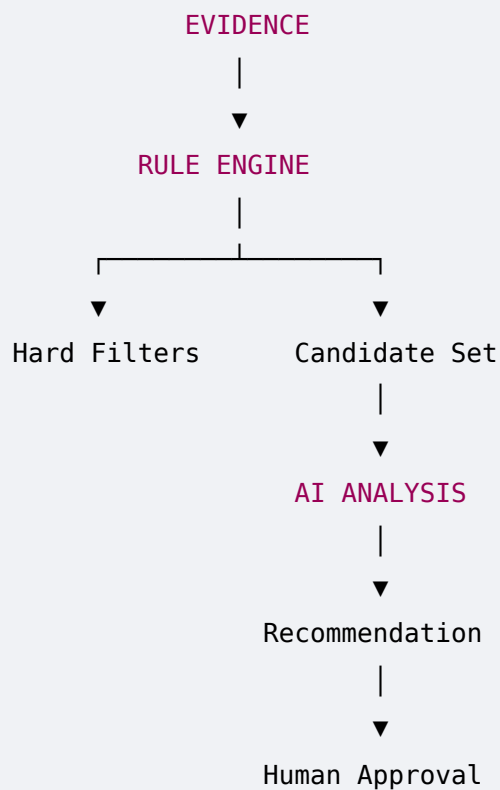
Bad AI task:

Invent a new cryptographic algorithm

Absolutely not.

56. AI + Deterministic Engine

Recommended architecture:



57. Crypto-Agility

Recommendation alone is insufficient.

Applications must be able to change algorithms in the future.

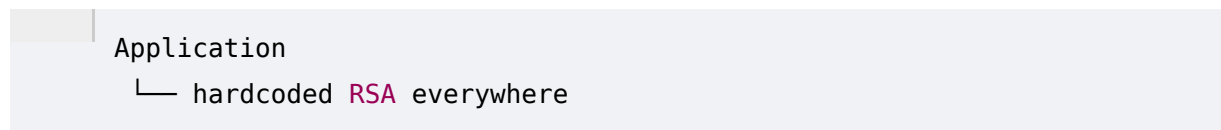
This is:

Crypto-Agility.

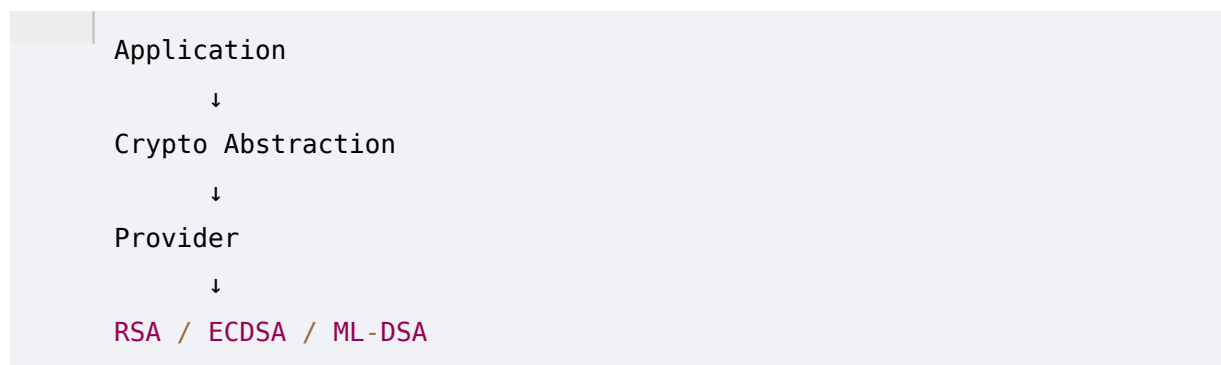
58. What Is Crypto-Agility?

An application is crypto-agile when it can replace cryptographic primitives without massive redesign.

Bad:

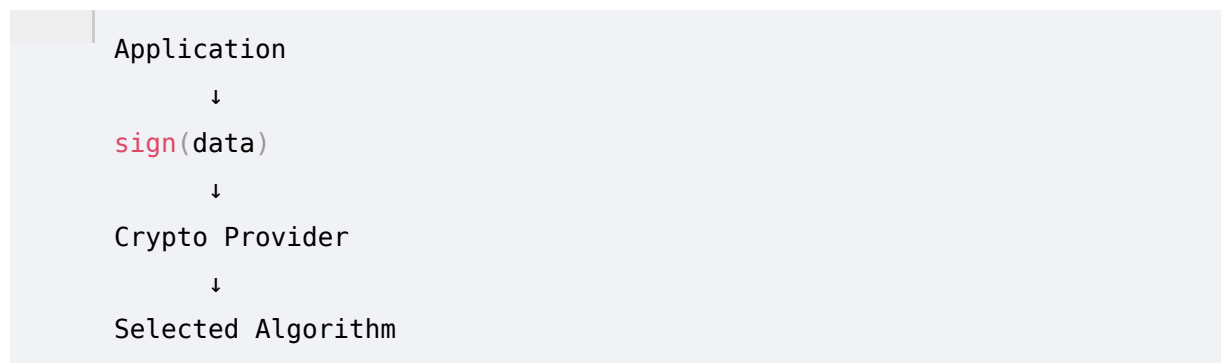


Better:



59. Crypto Abstraction Layer

Example:



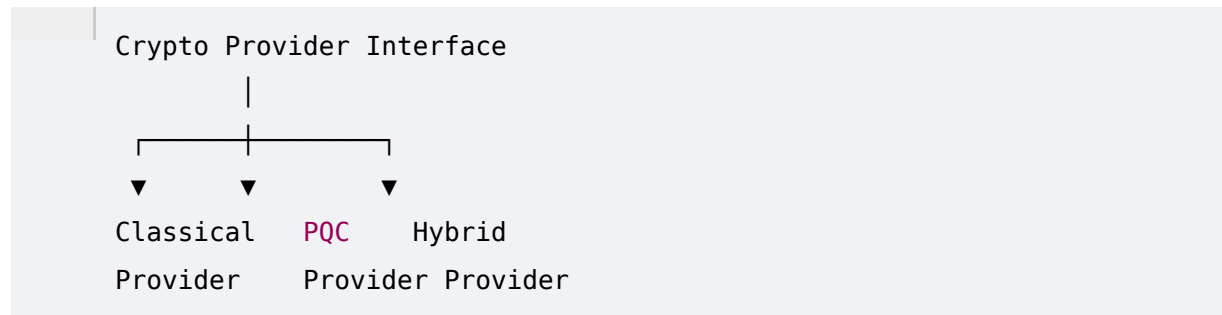
The application does not directly hardcode:



everywhere.

60. Provider Architecture

Conceptually:



This enables future algorithm replacement.

61. Algorithm Agnostic APIs

Bad:

```
ecdsa_sign(data)
```

Better:

```
sign(data, policy)
```

The policy decides which approved algorithm/provider to use.

62. Policy-Based Cryptography

Example:

```
{
  "purpose": "digital_signature",
  "minimum_security_level": 3,
  "quantum_ready": true,
  "allow_hybrid": true
}
```

The crypto layer selects the appropriate implementation.

63. Crypto Policy Engine

ECDAT can maintain organizational policies:

```
Approved
Conditional
```

Deprecated
Forbidden

Example:

RSA-1024:
FORBIDDEN

RSA-2048:
CONDITIONAL

ECDSA P-256:
MIGRATION_REQUIRED

ML-DSA:
APPROVED

64. Policy Enforcement

Policies can be applied during:

Development
CI/CD
Deployment
Runtime
Audit

65. CI/CD Example

Developer introduces:

RSA-1024

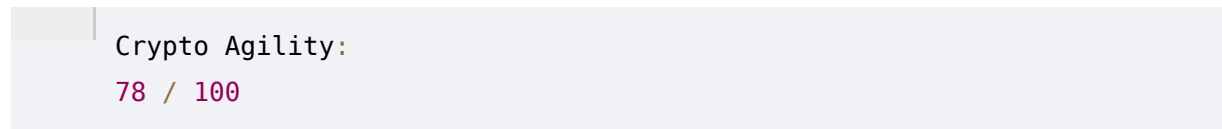
ECDAT:

Static Scan
↓
Policy Engine
↓
FORBIDDEN
↓
Build Failed

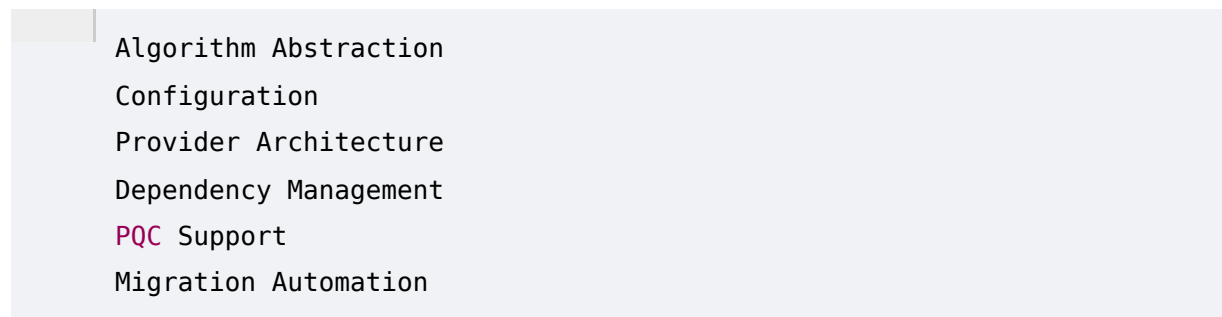
This prevents new technical debt.

66. Crypto-Agility Score

ECDAT could calculate:



based on:



This is a valuable enterprise metric.

67. Crypto-Agility Maturity

Example:

Level 0

Hardcoded cryptography.

Level 1

Central configuration.

Level 2

Crypto abstraction.

Level 3

Multiple providers.

Level 4

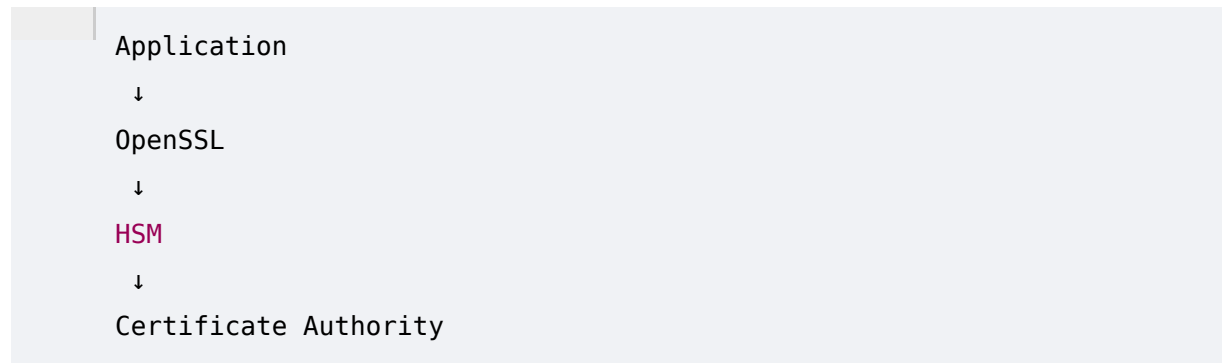
Automated algorithm migration.

Level 5

Continuous crypto lifecycle management.

68. Recommendation Dependency Graph

Suppose:



ECDAT might recommend:

1. Upgrade OpenSSL
2. Upgrade HSM
3. Establish PQC-capable PKI
4. Introduce hybrid protocol
5. Migrate application

This is much more useful than:

Replace ECDSA with ML-DSA.

69. Migration Sequence

The recommendation engine should produce:

- PHASE 1
Inventory
- PHASE 2
Upgrade dependencies
- PHASE 3
Enable PQC capability
- PHASE 4
Test hybrid mode
- PHASE 5
Deploy hybrid

PHASE 6

Monitor

70. Rollback

Every migration should have a rollback strategy.

Example:

```
PQC Deployment
  ↓
Monitoring
  ↓
Issue
  ↓
Rollback
  ↓
Classical / Hybrid
```

This should be part of migration planning.

71. Migration Verification

After migration:

```
Old Crypto:
ECDSA
```

should become:

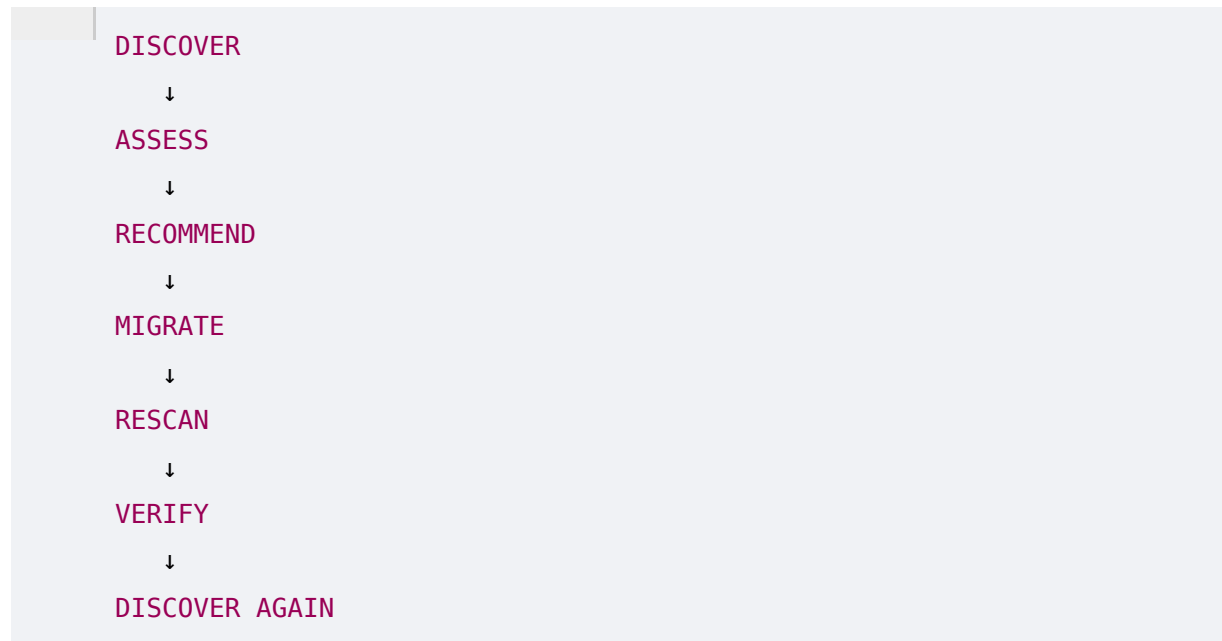
```
Current:
ML-DSA / Approved Hybrid
```

ECDAT rescans the environment.

```
Migration
  ↓
Rescan
  ↓
Compare
  ↓
Verify
```

72. Closed-Loop Migration

This creates:



This is a continuous security lifecycle.

73. Recommendation Example — ECDSA

Input:

```
Algorithm:
ECDSA P-256

Risk:
Critical

Application:
Government API

Library:
PQC-capable

HSM:
PQC support available

Clients:
Mixed
```


Possible output:

Primary:
Hybrid ECDSA + ML-DSA

Future:
ML-DSA

Alternative:
SLH-DSA

Reason:

Hybrid preserves interoperability
while introducing PQC protection.

74. Recommendation Example — RSA

Input:

RSA-2048

Use:
Code signing

Risk:
High

Consumers:
Legacy systems

Migration:
Complex

Recommendation:

Transition:
Dual-signing / hybrid strategy where supported

Long-term:
PQC signature migration

The exact mechanism depends on the signing ecosystem.

75. Recommendation Example — ECDH

Input:

	ECDH
Use:	Key establishment
Protocol:	TLS

Potential recommendation:

	PQC KEM migration
	or
	approved hybrid key establishment

The exact implementation must follow supported protocol standards.

76. Recommendation Example — AES-256

Input:

	AES-256-GCM
--	--------------------

Risk engine:

Quantum:	Lower relative concern
Migration:	Not necessarily urgent

Recommendation:

Maintain
+
Monitor
+
Ensure crypto-agility

This prevents unnecessary migration.

77. Recommendation Example — Custom Crypto

Input:

```
Custom Encryption
```

Recommendation:

```
DO NOT AUTOMATICALLY SUBSTITUTE
```

```
Action:
```

```
Security Review
```

```
Preferred:
```

```
Replace with standardized  
approved cryptography.
```

This is an important security principle.

78. Recommendation API

Example:

```
GET /api/assets/{asset_id}/recommendations
```

Response:

```
{  
  "asset": "ECDSA-P256",  
  "recommendations": [  
    {  
      "algorithm": "ML-DSA",  
      "strategy": "replacement",  
      "score": 0.91  
    },  
    {  
      "algorithm": "Hybrid",  
      "strategy": "transition",  
      "score": 0.96  
    }  
  ]  
}
```

```
]
}
```

79. Recommendation Data Model

A recommendation should include:

```
recommendation_id
asset_id
source_algorithm
target_algorithm
strategy
security_score
performance_score
compatibility_score
migration_score
cost_score
overall_score
confidence
rationale
constraints
created_at
```

80. Recommendation Explainability

The UI should show:

WHY?

Example:

Hybrid ML-DSA + ECDSA is recommended because the application has critical data, strict interoperability requirements, and legacy clients that cannot immediately support PQC-only authentication.

Then:

WHAT?

Introduce the hybrid mechanism.

Then:

HOW?

Upgrade the cryptographic provider, deploy compatibility testing, enable hybrid mode, monitor clients, and gradually retire the legacy path.

81. Recommendation Governance

AI-generated recommendations should be:

Suggested

not:

Automatically Applied

for critical systems.

Recommended states:

AI_SUGGESTED
RULE_VALIDATED
SECURITY_REVIEW
APPROVED
REJECTED
IMPLEMENTED
VERIFIED

82. Human-in-the-Loop

For high-risk systems:

ECDAT
↓
Recommendation
↓
Security Team
↓
Application Owner
↓
Approval
↓
Migration

This prevents blind automation.

83. Recommendation Audit Trail

Record:

```
Who generated it
Which model/rules were used
Which data was considered
What alternatives were rejected
Who approved it
When it was implemented
```

This is essential for enterprise governance.

84. Candidate Rejection Reasons

ECDAT should explain why candidates were rejected.

Example:

```
ML-DSA
✓ Security
✓ Library
x Legacy client compatibility
```

Another:

```
SLH-DSA
✓ Security
✓ Compatibility
x Signature size exceeds protocol constraint
```

This makes the recommendation engine much more credible.

85. Recommendation Confidence vs Algorithm Security

These must remain separate.

Example:

```
ML-DSA:
Security Confidence = High
```

Recommendation Confidence = Medium

because:

Application compatibility:
Unknown

The algorithm itself may be well understood, while the recommendation context is incomplete.

86. Recommendation Quality Metrics

ECDAT should measure:

Recommendation Acceptance Rate
Recommendation Rejection Rate
False Recommendation Rate
Migration Success Rate
Rollback Rate
Performance Deviation

These metrics can improve future recommendations.

87. Benchmarking

ECDAT should maintain benchmark data for:

Key Generation
Encapsulation
Decapsulation
Signing
Verification
Memory
Bandwidth

across supported environments.

For example:

Architecture:
x86_64

OS:

Linux

Library:

Version X

Hardware:

CPU Y

Benchmark data should be tagged with environment details.

88. Avoiding False Precision

Do not tell an organization:

ML-DSA will add exactly 0.73 ms.

unless ECDAT actually measured it in that environment.

Instead:

Estimated Performance Impact:

Low

Evidence:

Benchmark on comparable infrastructure

or:

Measured:

0.81 ms

when actually measured.

89. Recommendation Scenarios

ECDAT can provide:

Security-first

Maximize long-term security

Performance-first

Minimize latency

Compatibility-first

Minimize disruption

Cost-first

Minimize migration cost

Balanced

Balanced optimization

90. Enterprise Recommendation Dashboard

Example:

PQC MIGRATION ADVISOR	
Assets Requiring Migration:	1,284
Critical:	112
High:	391
Recommended Strategy:	
Hybrid	48%
Direct PQC	31%
Library Upgrade	14%
Reconfigure	5%
Decommission	2%

91. Application-Level Migration Plan

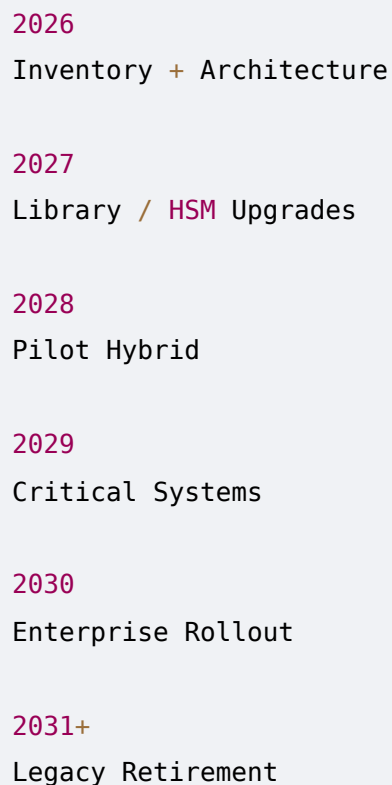
For each application:



- Application
- |
- Current Crypto
- Risk
- Target Crypto
- Dependencies
- Blockers
- Migration Steps
- Owner
- Timeline
- Validation
- Status

92. Migration Roadmap

ECDAT can produce:



- 2026
Inventory + Architecture
- 2027
Library / HSM Upgrades
- 2028
Pilot Hybrid
- 2029
Critical Systems
- 2030
Enterprise Rollout
- 2031+
Legacy Retirement

The dates should be based on actual organizational planning rather than invented automatically.

93. Migration Dependency Ordering

ECDAT should identify:

```
Infrastructure
```

```
↓
```

```
Crypto Library
```

```
↓
```

```
Protocol
```

```
↓
```

```
Application
```

```
↓
```

```
Clients
```

and recommend migrating prerequisites first.

94. Crypto-Agility Assessment

ECDAT can evaluate an application:

```
Algorithm hardcoded:
```

```
YES
```

```
Crypto abstraction:
```

```
NO
```

```
Provider interface:
```

```
NO
```

```
PQC support:
```

```
NO
```

```
Configuration driven:
```

```
NO
```

Result:

```
Crypto Agility:
```

```
Low
```

Recommendation:

```
Introduce crypto abstraction.
```

95. High-Agility Application

Example:

Algorithm abstraction:

YES

Provider architecture:

YES

Configurable:

YES

PQC provider:

YES

Hybrid:

YES

Result:

Crypto Agility:

High

Migration becomes easier.

96. Strategic Insight

The organization should not only migrate algorithms.

It should migrate architecture toward:

Crypto-Agility.

Otherwise:

RSA

→ PQC

today becomes:

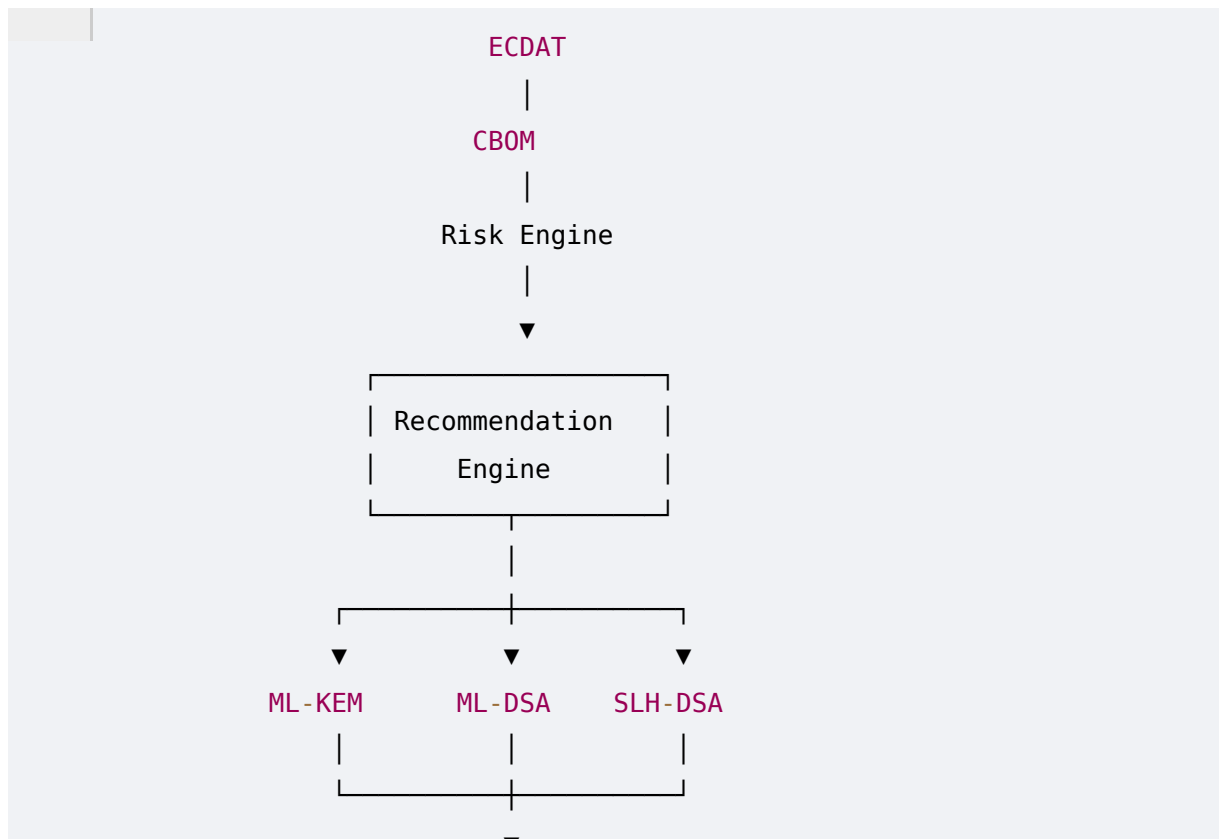
PQC Algorithm A

→ PQC Algorithm B

tomorrow, requiring another massive rewrite.

Crypto-agility prevents this.

97. Complete Phase 13 Architecture



98. Requirements Derived From Phase 13

The ECDAT prototype should support:

Algorithm Registry

- ML-KEM
- ML-DSA
- SLH-DSA
- Future algorithms

Recommendation

- Replacement
- Hybrid
- Parallel
- Upgrade
- Reconfiguration
- Decommission

Constraints

- Security
- Latency
- Throughput
- Memory
- Bandwidth
- Compatibility
- Hardware
- Libraries
- Protocols
- Cost

Governance

- Policy
- Approval
- Audit
- Explainability
- Confidence

Crypto-Agility

- Abstraction
- Providers
- Policy-driven selection
- Configuration
- Agility scoring

99. The Most Important Design Principle

ECDAT should **never** make the recommendation:

"Use ML-DSA everywhere."

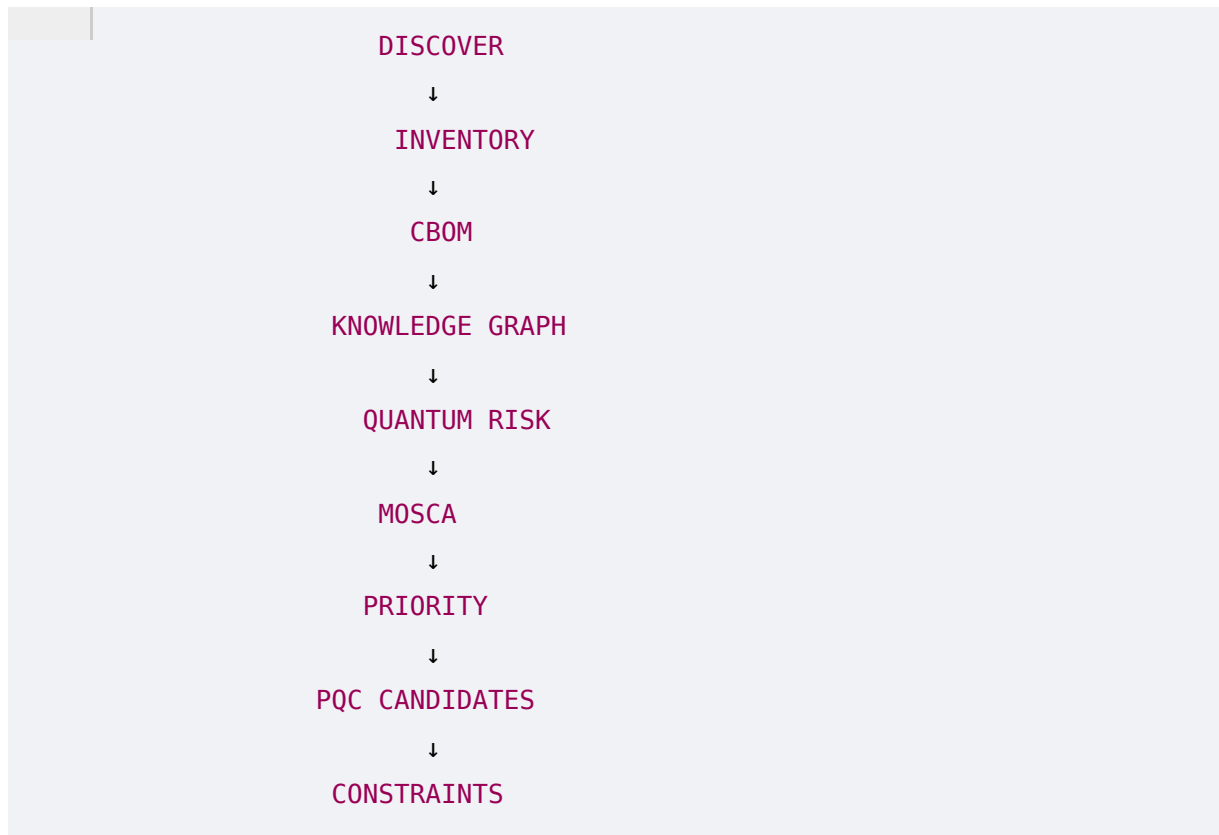
Instead:

"For this asset, under these constraints, this migration strategy provides the best balance of quantum security, interoperability, performance, cost, and migration feasibility."

That distinction is extremely important.

100. The ECDAT Decision Chain So Far

We now have:



This is beginning to look like a genuine enterprise platform rather than a simple SIH scanner.

101. What Makes This Different From a Normal Scanner?

A conventional security scanner says:

Found **RSA**.

ECDAT should say:

Found **RSA-2048**.

Used by:
Payment **API**

Protects:
Sensitive financial data

Quantum posture:
Vulnerable

Data lifetime:
12 years

Migration:
5 years

That is the **strategic value proposition** of ECDAT.

PHASE 14 PREVIEW — AI INTELLIGENCE LAYER, SEMANTIC CODE ANALYSIS & EXPLAINABLE CRYPTOGRAPHIC REASONING

At this point, the deterministic architecture is strong:

```
graph TD; A[Discovery] --> B[CBOM]; B --> C[Graph]; C --> D[Risk]; D --> E[Recommendation];
```

Discovery
↓
CBOM
↓
Graph
↓
Risk
↓
Recommendation

Now comes the AI layer.

The critical question is:

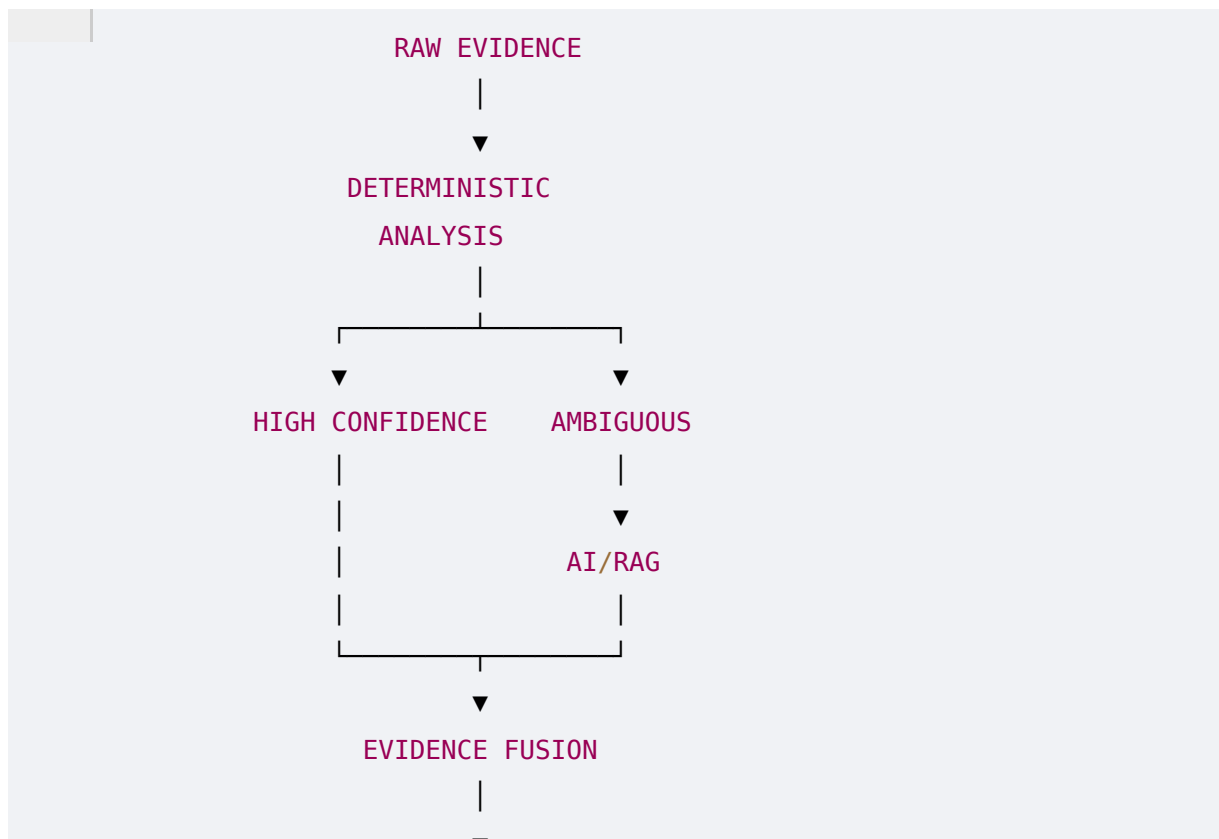
Where should AI actually be used, and where should it NOT be trusted?

Phase 14 will design the AI architecture covering:

- AI-assisted source-code analysis
- Semantic cryptographic detection
- Code understanding

- Custom crypto detection
- Ambiguous algorithm identification
- Configuration interpretation
- Dependency reasoning
- Context extraction
- Application architecture understanding
- Data sensitivity inference
- Migration explanation
- Recommendation explanation
- Natural-language querying
- RAG architecture
- Cryptographic knowledge base
- Retrieval pipeline
- Evidence-grounded generation
- Hallucination control
- Prompt injection protection
- AI confidence
- Human-in-the-loop
- AI vs deterministic engine boundaries
- Model selection
- Local/private models
- Air-gapped AI
- Model evaluation
- Red-team testing
- Explainability
- AI audit logs
- Cost control
- Token optimization
- Caching
- AI-assisted report generation
- Natural-language security analyst interface

The key architecture will be:



The goal is **not** to build an "AI scanner."

The goal is to build a **cryptographic intelligence system where AI is powerful precisely because it is constrained by evidence, deterministic analysis, and the canonical ECDAT model.**

Phase 13 complete.